

Riphah International University



Project Title: The Closet Fashion Assistant

(AI-Based Smart Outfit Recommendation System)

Submitted to: Ma'am Shumaila Qayum

Course: Artificial Intelligence

Semester: 06

Team Members:

Name	Sap I'd
Afifa Amna	55466
Anosha Pervaiz Awan	56200
Nimra Tariq	54909
Areeza Afridi	55254
Simab Malik	54910
Nighat Naseem	53740

Contents

1. Introduction:	3
2. High-Level System Flow	3
1. Percept Collection	3
2. Core Processing Pipeline	3
3. Action	3
3. Data Structures:	4
4. Core Algorithms:	4
4.1 Preprocessing & Filtering ($O(n)$)	4
4.2 Candidate Generation.....	4
4.3 Style Similarity using KNN ($k=5$).....	5
4.4 Utility Function (Multi-objective Scoring).....	5
4.5 Search & Selection: A* Heuristic Search	5
4.6 Learning Mechanism (Reinforcement-style Update).....	5
5. Pseudocode – Main Agent Function:	6
6. Implementation Technologies	6
7. Performance & Edge Cases	6
8. Summary.....	7

1. Introduction:

This document presents the detailed Algorithm Design Plan for *The Closet Fashion Assistant*. It builds directly on the Agent Design and Environment Analysis (utility-based agent) submitted previously. The plan specifies the exact algorithms, data structures, scoring mechanisms, and learning strategy that will be implemented to generate intelligent, personalized outfit recommendations in real time.

The design ensures multi-criteria optimization (weather, occasion, cleanliness, style coherence, and diversity) while keeping computational cost low for a smartphone app.

2. High-Level System Flow

Percept → Processing → Action

1. Percept Collection (Sensors)

- Real-time weather (API)
- User-selected occasion
- Clean clothes list + images/metadata from wardrobe database
- Wear history log

2. Core Processing Pipeline (Utility-based Agent)

- Preprocess & filter clothes
- Generate candidate outfit combinations
- Compute utility score for each combination using KNN + weighted scoring
- Select optimal outfit via heuristic search (A* variant)
- Learn from user feedback

3. Action (Actuators)

- Display outfit preview image
- Update dirty/clean status and history
- Offer “alternative” button

3. Data Structures:

Entity	Representation	Key Attributes
Clothing Item	Object / Dictionary	id, category (top/bottom/shoes), color (RGB + name), pattern, material, season_suitability, last_worn_date, wear_count, image_path
Outfit Combination	Tuple / Object (top, bottom, shoes)	items list + total_utility_score
Weather Data	Dictionary	temp, condition, season
Wear History	List of past outfits (last 30 days)	date, outfit_id, user_rating (optional)
User Preference	Vector (for KNN)	style_features (color harmony, pattern preference)

4. Core Algorithms:

4.1 Preprocessing & Filtering ($O(n)$)

- Fetch only **clean** clothes (filter by dirty flag).
- Tag each item with weather compatibility score (rule-based):
 - Temp $< 15^{\circ}\text{C}$ $\rightarrow$ warm layers (+1)
 - Temp $> 30^{\circ}\text{C}$ $\rightarrow$ light fabric (+1)
 - Rain $\rightarrow$ waterproof (+1)
- Occasion compatibility matrix (pre-defined lookup table): Example:
Wedding $\rightarrow$ formal shirt = 1.0, casual t-shirt = 0.2

4.2 Candidate Generation

- **Cartesian product** of clean tops $\times$ bottoms $\times$ shoes (limited to max 500 combinations for performance).
- Early pruning: discard any combination whose weather or occasion score is < 0.3 .

4.3 Style Similarity using KNN (k=5)

- Each clothing item is represented as a **feature vector** (one-hot encoded category + normalized color + pattern).
- Train a lightweight KNN model on:
 - User's past **accepted** outfits (positive examples).
- For every candidate outfit:
 - Compute average distance to the 5 nearest liked outfits.
 - Convert distance → **style_similarity_score** (0–1).

4.4 Utility Function (Multi-objective Scoring)

Each outfit receives a single **utility score** (0–1):

$$\text{Utility}(O) = w_1 \cdot \text{WeatherMatch} + w_2 \cdot \text{OccasionMatch} + w_3 \cdot \text{StyleSimilarity} + w_4 \cdot \text{Diversity}$$

Where:

- $w_1 = 0.35, w_2 = 0.30, w_3 = 0.20, w_4 = 0.15$ (weights tunable by user)
- **Diversity** = $1 - \frac{\text{recent_wear_count}}{\text{max_allowed}}$ (avoids repetition)

4.5 Search & Selection: A* Heuristic Search

- Model outfit selection as a **search problem** (states = partial outfits).
- **Heuristic $h(n)$** = estimated remaining utility (pre-computed max possible score).
- **Cost $g(n)$** = negative utility of current partial outfit.
- Goal = complete outfit with highest final utility.
- This is faster than pure brute-force when wardrobe size > 50 items.
- Fallback: if < 200 candidates → simple **argmax** on utility scores.

4.6 Learning Mechanism (Reinforcement-style Update)

- **After user accepts/rejects an outfit:**
 - If accepted → add to “liked” dataset for KNN retraining.
 - If rejected → increase penalty weight for that combination's features.
 - Update wear history and recalculate diversity scores.
- Periodic retraining (every 10 recommendations) using scikit-learn KNN.

5. Pseudocode – Main Agent Function:

```
function RecommendOutfit(weather, occasion, clean_clothes, history):  
    filtered = FilterCleanClothes(clean_clothes)  
    candidates = GenerateCombinations(filtered)           // limited Cartesian  
product  
  
    for each outfit in candidates:  
        weather_score = ComputeWeatherMatch(outfit, weather)  
        occasion_score = ComputeOccasionMatch(outfit, occasion)  
        style_score = KNN_StyleSimilarity(outfit, history.liked_outfits)  
        diversity_score = ComputeDiversity(outfit, history)  
  
        outfit.utility = 0.35*weather_score + 0.30*occasion_score  
                        + 0.20*style_score + 0.15*diversity_score  
  
    best_outfit = A_Star_Search(candidates)  
  
    DisplayOutfitPreview(best_outfit)  
    return best_outfit
```

6. Implementation Technologies

- Language: Python (Flask/FastAPI backend + React Native mobile app)
- ML Library: scikit-learn (KNN)
- Search: Custom A* implementation (or NetworkX)
- Weather API: OpenWeatherMap (free tier)
- Database: SQLite / Firebase (wardrobe + history)
- Image Handling: Pillow + simple metadata tagging (future: CLIP model for auto-tagging)

7. Performance & Edge Cases

- Time complexity: $O(n \times k)$ where n = clean items, k = small constant (target < 2 seconds on mobile).
- Edge cases handled: Empty wardrobe → “Laundry mode” prompt; extreme weather → strict filtering; first-time user → default weights.
- Evaluation Metrics: Precision@3 (user acceptance rate), average utility score, repetition rate (<10%)

8. Summary

The algorithm design transforms the utility-based agent into a fully functional, efficient recommendation engine. KNN ensures style personalization, the weighted utility function handles multi-criteria optimization, and A* provides intelligent search. The learning component makes the system adaptive over time — exactly as required for a real-world smart closet assistant.

This plan is ready for coding in the next phase. All algorithms align with the PEAS description and environment properties defined earlier.